Subtyping

Subtyping is a cross-cutting concept that relates to many language features in non-trivial ways. It is characteristically found in object-oriented languages and is often considered an essential feature of the object-oriented style. Though we will explore this connection in detail, subtyping is just as relevant to functional languages.

Among the languages of this course, Scala combines subtyping with parametric polymorphism in remarkably expressive ways: bounded type parameters, variance annotations on type constructors, and a careful interaction between the two let the programmer state precisely how a generic type relates to its parameters. This expressiveness comes at a price, however: Scala requires the programmer to write explicit type annotations in many situations where, in principle, the compiler could infer them.

Haskell goes the opposite way: it does not have subtyping at all, and instead leverages parametric polymorphism and type classes (which we will discuss later) to recover much of the same expressive power. The reward for this restriction is one of the most powerful type inference engines in any mainstream language (Haskell programs are typically free of type annotations almost entirely). The two languages thus represent two complementary design points: subtyping with annotations, or no subtyping with inference.

Subsumption and the substitution principle

Without subtyping, the typing rules for function application are unnecessarily rigid. The requirement that the argument types exactly match the function domain type leads the type-checker to reject many programs that are perfectly well-behaved. Recall the typing rule for function application:

$$\frac{\Gamma \vdash M : S \to T \qquad \Gamma \vdash N : S}{\Gamma \vdash M\, N : T} \quad (\text{T-App})$$

According to this rule, the well-behaved term $(\lambda r : \{x : \mathsf{Int}\}.(r.x))\{x = 0, y = 1\}$ is ill typed, since the type of the argument is $\{x : \mathsf{Int}, y : \mathsf{Int}\}$, whereas the function accepts $\{x : \mathsf{Int}\}$.

But, clearly, the function just requires that its argument be a record with a field x; it doesn't care what other fields the argument may or may not have. This is the case for this specific function, as we readily realize that it is always safe to pass an argument of type $\{x : \mathsf{Int}, y : \mathsf{Int}\}$ to a function that expects $\{x : \mathsf{Int}\}$. T

We capture this intuition, and generalize it, by formalizing the *principle of safe substitution* : we say that S is a subtype of T (or T is a supertype of S) written $S <: T$, to mean that any term of type S can safely be used (or *substituted*) in a context where a term of type T is expected.

The principle of substitution can be implemented in the typing systems my adding a new typing rule to connect the typing and subtyping relations, as shown below by what is commonly referred to as the rule of *subsumption*:

$$\frac{\Gamma \vdash M : S \qquad S <: T}{\Gamma \vdash M : T} \ (\text{T-Sub})$$

This rule tells us that, if $S <: T$, then every term M of type S is also a term of type T.

To exemplify, consider our motivating example above.

If we define the subtype relation so that $\{x : \mathsf{Int}, y : \mathsf{Int}\} <: \{x : \mathsf{Int}\}$, then we can use rule (T-SUB) to derive $\Gamma \vdash \{x = 0, y = 1\} : \{x : \mathsf{Int}\}$, which is what we need to make the example type-check. In fact, if we assume that $\{x : \mathsf{Int}, y : \mathsf{Int}\} <: \{x : \mathsf{Int}\}$ we can construct the following derivation:

$$\dfrac{\dfrac{\Gamma, r : \{x : \mathsf{Int}\} \vdash r.x : \mathsf{Int}}{\Gamma \vdash (\lambda(r : \{\mathsf{Int}\}).r.x : \{x : \mathsf{Int}\} \to \mathsf{Int}} \ (\text{T-Abs}) \qquad \dfrac{\Gamma \vdash \{x = 0, y = 1\} : \{x : \mathsf{Int}, y : \mathsf{Int}\} \quad \{x : \mathsf{Int}, y : \mathsf{Int}\} <: \{x : \mathsf{Int}\}}{\Gamma \vdash \{x = 0, y = 1\} : \{x : \mathsf{Int}\}} \ (\text{T-Sub})}{\Gamma \vdash (\lambda(r : \{x : \mathsf{Int}\}).r.x)\{x = 0, y = 1\} : \mathsf{Int}} \ (\text{T-App})$$

The subtype relation

By referring to *safety*, in the substitution principle, we mean to say that the substitution of a subtype for a type must preserve the soundness of the type system, i.e. the type system's ability to ensure the absence of run-time type errors.

The subtype relation is formalized as a system of inductive rules deriving judgements of the form $S <: T$.

We'll introduce one or more rules for each kind of type (function types, record types, . . . etc.). However, before we get to the rules for particular type constructors, we introduce two general principles.

Firstly, subtyping should be reflexive and transitive,

$$\frac{}{T <: T} \ (\text{S-Refl}) \qquad \frac{S <: U \qquad U <: T}{S <: T} \ (\text{S-Trans})$$

These rules clearly support the principle of safe substitution (which is itself reflexive and transitive).

An intuitive guide to formalizing the subtype relation on the specific kinds of types, one which also provides a justification for the principle of safe substitution, may then be given according to the two possible readings of $S <: T$:

- Operational: "*every operation that is legal on values of type T is legal on values of type S*" ;
- Set theoretic: "*every value described by S is also described by T,*" that is, domain (S) ⊆ domain (T)

Record subtyping

We have already seen that we may consider $S = \{\ell_1 : T_1, \ldots, \ell_m : T_m\}$ to be a subtype of T if T is a record type with fewer fields than S. In particular, it is safe to "forget" some fields at the end of a record type. The so-called *width subtyping* rule, below, captures this intuition:

$$\frac{}{\{\ell_1 : T_1, \ldots, \ell_{m+k} : T_{m+k}\} <: \{\ell_1 : T_1, \ldots, \ell_m : T_m\}} \quad \text{(S-Rec-W)}$$

While it may at first seem surprising that the *smaller* type (the subtype) is the one with more fields, it is instructive to adopt a refined intuition of record types than the one we proposed earlier.

Specifically, we should regard a *record type $\{\ell_1 : T_1, \ldots, \ell_m : T_m\}$ as describing *the set of all records containing **at least** the fields $\ell_1, \ldots, \ell_m$ at their respective types*. In other words, a longer record constitutes a more demanding - i.e., more informative - specification, and so describes a smaller set of values.

According to this view, records like $\{x = 3\}$ and $\{x = 5\}$ are clearly elements of the domain of $\{x : \mathsf{Int}\}$, but so are values like $\{x = 3, y = 100\}$ and $\{x = 3, a = \mathsf{true}, b = \{y = 5\}\}$ as well.

Similarly, the record type $\{x : \mathsf{Int}, y : \mathsf{Int}\}$ describes all records with at least the fields x and y, both of type Int. Values like $\{x = 3, y = 100\}$ and $\{x = 3, y = 100, z = true\}$ are members of this type, while $\{x = 3\}$ is not, and neither is $\{x = 3, a = \mathsf{true}, b = \mathsf{true}\}$.

The width subtyping rule applies only to record types whose common fields are identical. In fact, if the record fields are *immutable* (as we have assumed so far), it is safe to allow the types of individual fields to vary from a type to its supertypes, as long as the types of each corresponding field are in the subtype relation. The *depth subtyping* rule expresses this intuition:

$$\frac{S_i <: T_i \quad (i = 1, \ldots, m)}{\{\ell_1 : S_1, \ldots, \ell_{m+k} : S_{m+k}\} <: \{\ell_1 : T_1, \ldots, \ell_m : T_m\}} \quad \text{(S-Rec)}$$

Function Subtyping

The subtype relationship between functions is trickier.

Suppose some context expects a function of type $S \to T$. We can represent this as the following Scala definition:

```
def use(f: S => T): ... =
  ...                 // The body will call f with values of type S
                      // and use the resulting T values
```

The question we want to answer is can we safely substitute a function of type $S' \to T'$ for one of type $S \to T$ (because the former type is a subtype of the latter)?

The return type. Inside `use`, the result of calling `f` is treated as a T value. If our function actually returns T' values, the substitution is safe whenever $T' <: T$, and `use` never sees the difference. So we may *narrow* the return type in subtypes. Function types are **covariant in the return type**.

The argument type. Inside `use`, `f` is called on values of type S. The function we pass for f will be receiving these S values. The substitution is safe whenever our function can handle any S — that is, whenever it has type $S' \to T$, with $S <: S'$. So we may *widen* the argument type in subtypes. Function types are **contravariant in the argument type**.

Combining the reasoning, we have the following subtype rule:

$$\frac{S <: S' \qquad T' <: T}{S' \to T' <: S \to T} \ (\text{S-Arrow})$$

Or more memorably: a function may *accept more and return less* than the type it replaces.

Subtyping and Inheritance in Object-Oriented Languages

In mainstream Object-Oriented languages, the subtype relation is established explicitly via the *class extension* and the *interface implementation* mechanisms: class and interface are types, and a class `A` declares to extend another class (to implement an interface) `B` to be subtype of `B`.

Structural similarity between two classes in not enough to activate the subtype relationship: two classes with identical fields but different names are different types. This is nominal typing, the discipline familiar from Java and adopted widely in OO languages (including Scala).

The structural subtyping rules from the previous section are still in force, but they play a different role: instead of determining whether `B <: A`, they verify whether a declared relation `class B extends A` is safe, by checking that common / overriding methods have compatible signatures.

While tigtly bound by such mechanisms, inheritance and subtyping are conceptually different and serve two very different purposes, associated with two distinct code-reuse mechanisms in object-oriented languages:

- **Inheritance and class extension** enable the reuse of *implementation* code: subclasss need not repeat unchanged fields and methods of their superclass — they inherits them.
- **Subtyping** enables the reuse of *client* code: client code written against a supertype needs not change when new subclasses are added; the existing code (which mentions only the supertype) continues to work.

These two roles are independent. A language could in principle support subtyping without inheritance (purely structural systems work this way) or inheritance without subtyping (rare, but conceivable). In Scala, as in most object-oriented languages, the two come together: the same `extends` keyword establishes both the inheritance relationship and the subtype relationship.

Scala Traits as specifications

Scala's primary mechanism for expressing subtype hierarchies is the *trait*. We have already encountered traits in the data type design lecture, where we used `sealed trait` together with case classes to define algebraic data types. We now look at traits more systematically.

Syntactically, a trait is defined like a class but with the keyword `trait`. A trait may include both *abstract* method declarations (signatures only) and *concrete* method implementations. Like abstract classes, traits cannot be instantiated directly — but unlike abstract classes, a class may extend (or "mix in") *multiple* traits.

In its simplest use, a trait serves as a pure specification: a list of method signatures that implementing classes must provide:

```
sealed trait Shape:
  def area: Double
  def scale(f: Double): Shape

case class Circle(radius: Double) extends Shape:
  def area: Double = math.Pi * math.pow(radius, 2)
  def scale(f: Double): Shape = Circle(radius * f)

case class Rectangle(base: Double, height: Double) extends Shape:
  def area: Double = base * height
  def scale(f: Double): Shape = Rectangle(base * f, height * f)
```

The trait is `sealed` — meaning that no further extensions are permitted outside this file — which makes it both a specification and a closed device for pattern matching. Each implementing case class provides the concrete bodies for `area` and `scale`. Client code can be written against `Shape` and remain ignorant of which concrete subtype it is dealing with.

This use of traits is essentially the same as Java's pre-8 interfaces: a list of abstract methods that subclasses are required to implement. Scala traits go further, however — they may carry *implementations*, which is where they become genuinely interesting.

Overriding and dynamic dispatch

A central mechanism that makes subtyping useful — and that connects subtyping to inheritance — is *method overriding*. A subclass may provide its own implementation of a method declared in the supertype, replacing the inherited one. In the `Shape` hierarchy above, both `Circle` and `Rectangle` *override* the abstract `area` method from `Shape`, each providing its own implementation tailored to its geometry.

In the following alternative hierarchy, the `Shape` trait provides with a default description method, and the two subclasses explicitely override it.

```
sealed trait Shape:
  def area: Double
  def scale(f: Double): Shape
  def describe: String = s"a shape with area $area"     // concrete

case class Circle(radius: Double) extends Shape:
  def area: Double = math.Pi * math.pow(radius, 2)
  def scale(f: Double): Shape = Circle(radius * f)
  override def describe: String = s"a circle of radius $radius"   // overrides

case class Rectangle(base: Double, height: Double) extends Shape:
  def area: Double = base * height
  def scale(f: Double): Shape = Rectangle(base * f, height * f)
  // inherit the default definition
```

What makes overriding effective is *dynamic dispatch*: when a method is invoked on an object, the implementation that gets executed is determined by the object's *runtime* (dynamic) type, not by the *static* type of the variable holding it. Consider:

```
def report(s: Shape): String = s.describe

val c: Circle    = Circle(5.0)
val r: Rectangle = Rectangle(3.0, 4.0)

report(c)    // uses Circle's override "a circle of radius 5.0"          –
report(r)    // "uses Shape's default shape with area 12.0"          –
```

Both `c` and `r` are statically typed as `Shape` (via subtyping), so at compile time the call `s.describe` is checked against `Shape`'s contract. At runtime, however, the implementation that executes depends on the actual object: `c` is a `Circle`, so `Circle.describe` runs; `r` is a `Rectangle`, which inherits `describe` from `Shape`, so `Shape.describe` runs.

This combination — overriding plus dynamic dispatch — is what lets client code, written against a supertype, take advantage of subtype-specific behaviour without needing to know which subtype is involved. It is the core mechanism behind the "subtyping enables client-code reuse" slogan: a function written once against `Shape` works correctly for `Circle`, for `Rectangle`, and for any future `Shape` subtype — including those that override existing behaviour and those that simply inherit it.

A note on override compatibility

When a subclass overrides a method, the function-subtyping rule determines what signatures are compatible. In particular, the override may *return a more specific type* (covariance in the return) and, in principle, *accept a more general parameter type* (contravariance in the parameter), since either change preserves substitutability.

Of these, **covariant return types** are widely supported and useful. Suppose we add a `duplicate` method to our `Note` hierarchy:

```
sealed trait Note:
  def text: String
  def header: String = "Note"
  def duplicate: Note

class Reminder(val text: String, val date: String) extends Note:
  override def header: String = s"Reminder for $date"
  override def duplicate: Reminder = Reminder(text, date)   // narrower return type
```

The supertype's `duplicate` promises a `Note`; the `Reminder` override returns a `Reminder`. The override can safely substitute the superclass method — any caller expecting a `Note` is still served — and the narrower return type carries useful information for code that already knows it is dealing with a `Reminder`:

```
val r: Reminder = new Reminder("milk", "Mon")
val r2: Reminder = r.duplicate   // direct: returns a Reminder

val n: Note = r
val n2: Note = n.duplicate       // also fine: Reminder is a Note
```

Contravariant parameter types are permitted by the theory but rarely useful in practice. If a supertype's method takes a `Note`, an override could in principle widen the parameter to `Any` while remaining a safe substitute. But the implementer of the override almost always needs at least the original parameter type to do useful work — for example, the body might call `other.text`, which is not available on `Any`.

Languages tend not to bother supporting this rarely-useful flexibility: Scala identifies overrides by *exact* parameter type match, treating a method with a different parameter type as an *overload* rather than an override. To express genuine flexibility in parameter types, one uses *bounded parametric polymorphism* — a topic we will treat in the next lecture.

Designing interfaces

When designing an interface, two classes of users have *conflicting expectations*:

- **Implementers** of the interface — the classes that provide concrete definitions for its methods — would like the interface to be *thin*, to minimise their implementation effort.
- **Clients** of the interface — the code that calls its methods — would like the interface to be *rich*, with as many useful operations as possible.

Mixing implementation and specification within the same trait makes it possible to strike an adequate balance between these conflicting demands. A rich interface can be defined by *making most methods concrete*, with

their implementations expressed in terms of a small set of *abstract methods* left for implementers to provide. Implementers see only the small abstract surface; clients see the full rich interface.

A canonical example is the `Ordered` trait inspired by the corresponding type in the `scala.math` standard library:

```
trait Ordered[A]:
  // The abstract method — to be defined by implementers.
  def compare(that: A): Int

  // Concrete methods, defined in terms of compare.
  def <  (that: A): Boolean = this.compare(that) <  0
  def <= (that: A): Boolean = this.compare(that) <= 0
  def >  (that: A): Boolean = this.compare(that) >  0
  def >= (that: A): Boolean = this.compare(that) >= 0
```

The trait declares a single abstract method, `compare`, while the four comparison operators are provided as concrete methods that depend on it. Implementers need only define `compare`; clients have a full suite of comparison operators at their disposal.

This pattern — *one abstract method, many concrete methods* — is one of the most useful patterns in object-oriented design. The Java standard library adopted a similar approach with default methods in interfaces (introduced in Java 8, partly inspired by Scala traits): the conceptual story is the same, with minor syntactic differences.

Beyond subtyping: traits, mix-ins, and modular composition

The use of traits illustrated in this lecture — as specification devices and as carriers of default implementations — exercises the subtyping dimension of the trait mechanism: a class that extend a trait becomes a subtype of that trait, and clients written against the trait can use any such class.

Scala traits, however, support a second, distinctly different use: as modifiers that can be combined with a class on the fly to alter its behaviour. So far we have used traits in declarations such as class `Circle extends Shape`, where the relationship between the class and the trait is fixed at definition time.

Scala also allows traits to be combined (*mixed in*) with a class at instantiation time, with the syntax `new BaseClass with TraitA with TraitB`. The expression creates a new object whose behaviour combines `BaseClass` with the modifications introduced by `TraitA` and `TraitB`.

The mechanism that makes this work is the dynamic binding of `super`: in a trait, the meaning of `super.m(...)` is not fixed at the point of definition but is resolved at the point where the combination occurs. The same trait can therefore be combined with different classes and behave correctly in each.

This is the basis of Scala's *stackable modifications*, where the order of the traits in the combination — through a process called linearization — determines the resulting behaviour. Traits used this way enable modular, reusable, composable code that goes well beyond what classical inheritance provides.

This second role of traits — as units of modular composition rather than of subtype declaration — connects to a broader area of programming-language design: component-based development and mix-in composition.

The questions there are different from those raised by subtyping: how does one assemble large systems from independently developed parts, how are conflicts resolved when components contribute overlapping behaviour, what guarantees does composition preserve? These questions are largely orthogonal to substitutability and the Liskov principle, and the language mechanisms that address them — *mixins*, multiple-inheritance disciplines, linearization, . . . — form a research and design tradition of their own. Interested readers may consult the chapters on traits and mixins composition in Programming in Scala, or any treatment of feature-oriented programming and component-based software engineering. We will not pursue this thread further in this course.

REFERENCES

The Scala material is based on Chapter 11 of *Programming in Scala (5th Edition)*, M. Odersky, L. Spoon, B. Venners, F. Sommers, Artima Press, 2021.